TASK 1 — STEP-BY-STEP GUIDE

ANALYZE THE SOFTWARE SPECIFICATION

Course: iOS Programming | Week 5 | Suggested time: 20 minutes | Individual work

1. Learning Outcomes

- Distinguish functional requirements from non-functional requirements.
- Extract actions, data, constraints, and quality expectations from a written specification.
- Write at least 8 testable functional requirements and 4 measurable non-functional requirements.
- Identify potential user roles and explain role assumptions.

2. The Problem Statement — Read First

A school needs a small iOS application to manage students, subjects, teachers, and student enrollments for academic year 2026–2027. The school maintains records of students (student ID, name, email), teachers (teacher ID, name, email), and subjects (subject ID, title, credits, semester). A teacher may teach multiple subjects; a subject may be taught by more than one teacher. Students may enroll in multiple subjects. Each enrollment records its academic year and a mark from 0 to 10. A student must not be enrolled twice in the same subject for the same academic year. The app allows staff to browse records, add and edit students, assign subjects to teachers, enroll students, enter or update marks, and inspect a student's subjects and marks. Records must be understandable and validated. The app is used on an iPhone and should remain responsive with at least 100 sample enrollments.

Important: Do NOT start designing Swift classes or screens. Your task is to describe WHAT the system must do, not HOW it will be implemented.

3. Guided Procedure

Step 1 — Read and Annotate (4 minutes)

Read the specification twice. On the second pass, use four labels: N = nouns/data, V = verbs/actions, B = business rules, Q = quality constraints.

Label	What to look for	One example
N	Information or real-world objects	student ID
V	Actions performed by the app	browse records
B	Rules that restrict allowed operations	mark must be from 0 to 10
Q	Expected quality or operating conditions	responsive with 100 enrollments

Your turn: find at least 5 additional nouns, 5 actions, 2 rules, and 1 quality constraint. Record them below.

Nouns / data	Actions	Business rules	Quality constraints
Teacher ID, name, email	Add and edit students	No duplicate enrollment for the same student, subject and academic year	Must stay responsive with at least 100 sample enrollments
Subject ID, title, credits, semester	Assign subjects to teachers	A teacher may teach several subjects; a	Used on an iPhone

		subject may have several teachers	
Enrollment: academic year and mark	Enroll students in subjects	A student may enroll in several subjects	Records must be understandable and validated
Academic year 2026–2027	Enter or update marks	Each enrollment records one academic year and one mark	—
Staff (user of the app)	Inspect a student's subjects and marks	—	—

Step 2 — Write Functional Requirements (6 minutes)

Functional requirements describe behavior. Use the pattern: “The app shall + observable action + object + optional condition.” Give every requirement a unique identifier (FR-01, FR-02, ...).

Worked example: FR-01 — The app shall display a list of students showing each student’s ID and name.

Check: Can a tester observe whether the list appears and whether both fields are visible? Yes.

Avoid: “The app shall use a Student class and List.” This is an implementation decision, not a functional requirement.

ID	The app shall...	How to verify?
FR-01	Display students with ID and name.	Open the student list and inspect a record.
FR-02	Filter the student list by student ID or name.	Type “S003”; only that student remains.
FR-03	Add a student with student ID, name and email.	Add S026; the new row appears in the list.
FR-04	Edit an existing student's name and email.	Change the email; the detail screen shows the new value.
FR-05	Display subjects (ID, title, credits, semester), teachers, and the teachers of each subject.	Open a subject; check its fields and its teachers.
FR-06	Assign a subject to a teacher and remove an assignment.	Assign MAT101 to T01; it appears under T01.
FR-07	Enroll a student in a subject for an academic year.	Enroll S026 in IOS101 for 2026-2027; the record is listed.
FR-08	Reject a duplicate enrollment (same student, subject and academic year) and explain why.	Repeat FR-07; an error is shown and no new record is added.
FR-09	Enter or update the mark of an enrollment.	Enter 8.5; the mark shows 8.5.
FR-10	Reject marks outside 0–10 (or non-numeric) and keep the previous mark.	Enter 11; an error is shown and the old mark is kept.
FR-11	Display a student's enrolled subjects, marks and average mark.	Open a student; subjects, marks and average are visible.

FR-12	Validate required fields, email format and unique IDs, and show a message for each error.	Save with an empty name or an existing ID; a message is shown.

Add more rows if needed. Ensure you cover records, assignments, enrollment, marks, and validation without writing the implementation.

Step 3 — Write Measurable Non-Functional Requirements (5 minutes)

Non-functional requirements describe quality or constraints. Use a measurable acceptance criterion rather than vague words such as “fast,” “simple,” or “secure.”

Worked example (proposed measurable interpretation): NFR-01 — With 100 sample enrollments, the student list shall become interactive within 2 seconds on the selected test simulator. This 2-second threshold is an exercise assumption, not a number stated in the specification.

ID	Quality attribute	Requirement and measurable acceptance criterion
NFR-01	Performance	With 100 enrollments, student list interactive within 2 s (exercise assumption).
NFR-02	Reliability	100% of invalid operations (invalid mark, duplicate enrollment, invalid student data) leave the stored data unchanged.
NFR-03	Usability	Every validation error appears as a readable text message on the same screen, without closing the form; enrolling a student takes at most 4 taps (my proposed criterion).
NFR-04	Compatibility	The app runs on iPhone devices/simulators with iOS 17 or later and is built with Xcode 15.
NFR-05	Consistency	After any add/edit/enroll/mark action, returning to a list shows the new value immediately, without restarting the app.

Possible categories to consider: usability, reliability, compatibility, performance, accessibility. Clearly label any numerical threshold you introduce as your own proposed criterion.

Step 4 — Identify Users and Permissions (3 minutes)

The specification explicitly mentions “staff” but does not define separate roles. Propose two plausible roles OR justify one staff role. Clearly mark assumptions rather than claiming they are specified.

Role	Tasks / permissions	Specified or assumed?
Role A: School Staff	Browse all records, add/edit students, enroll students, enter/update marks, inspect results	Specified

Role B: Academic Coordinator	Same as staff, plus assign subjects to teachers	Assumed (specialization of staff)

Role justification (2–3 sentences):

The specification only mentions “staff”, so one staff role with access to every function is enough for this app; no login is required in the lab scope. The Academic Coordinator is only a proposed specialization for assigning subjects to teachers; it is an assumption, not a stated requirement, and is not enforced by the app.

Step 5 — Review and Submit (2 minutes)

For every FR, ask: Is it observable? Is it one clear behavior? Does it trace to the specification?

For every NFR, ask: Is it measurable? How would I test it? Did I distinguish my own assumption from the original text?

4. Student Deliverables

☐ Annotated specification or a four-column extraction table (N/V/B/Q).

☐ A table with at least 8 functional requirements (FR IDs).

☐ A table with at least 4 non-functional requirements (NFR IDs and measurable criteria).

☐ One- or two-role proposal with justification and explicit assumptions.

Save your work as Task1_Requirements_StudentID.pdf. Include it later in Requirements.pdf for the full Lab 01 submission.

5. Self-Assessment Checklist

☑ I have at least 8 FRs, each beginning “The app shall…”.

☑ I have at least 4 NFRs, each with a testable acceptance criterion.

☑ I did not confuse UI widgets, Swift classes, or Core Data with requirements.

☑ I identified constraints: marks 0–10 and no duplicate enrollment per student, subject, and academic year.

☑ I identified user roles and labeled assumptions.

☑ Every requirement has a unique ID and can be verified.

6. Common Mistakes

Weak statement	Better approach
“The app must be fast.”	Define workload, device/test environment, and measurable response time.
“Create a Student.swift file.”	Specify the behavior involving student records, not a file name.
“Only teachers can edit marks.”	This permission is not specified; present it as a proposed assumption.
“The app must store data in Core Data.”	Persistence technology is not required for this in-memory lab.

Reflection: Which sentence in the specification was hardest to translate into a testable requirement, and why? (3–5 sentences)

The hardest sentence was “Records must be understandable and validated”, because “understandable” is subjective and cannot be tested directly. I translated it into observable behaviour: every record shows its ID and a readable name, and every invalid input produces a specific error message (FR-12, NFR-03). “Validated” was split into separate testable rules required fields, email format, unique IDs, mark range 0–10 and no duplicate enrollment so that each rule can be checked with its own test case.